

Návrh a implementace databázových systémů

Pokročilejší rysy jazyka SQL

PostgreSQL

- dokumentace
 - <http://www.postgresql.org/docs>

Datové typy PostgreSQL

- široká škála nativních datových typů
 - číselné (numeric)
 - znakové (character)
 - binární (binary)
 - datum a čas (date and time)
 - boolean
 - výčtové (enumerated)
 - geometrické (geometric)
 - pole (arrays)
 - kompozitní datové typy (composite types)
 - řetězce bitů (bit strings)
 - a další... (síťové adresy, XML, měnové, OID,...)
- lze přidávat další datové typy

Datové typy PostgreSQL

- číselné
 - celočíselné typy různé délky
 - `integer`
 - `serial` – automatické číslo
 - čísla s plovoucí řádovou čárkou
 - `real`
 - speciální hodnoty Infinity, -Infinity a NaN
 - desetinná čísla s pevnou řádovou čárkou
 - `decimal (totéž co numeric)` – velmi pomalé operace, přesnost až 1000 cifer

Datové typy PostgreSQL

■ znakové

- `char(n)` – řetězec délky přesně n
- `varchar(n)` – řetězec proměnné délky, max n
- `text` – řetězec neomezené délky
- char je v případě potřeby doplněn zprava na stanovenou délku mezerami
 - tyto mezery nejsou signifikantní, tj. neberou se v potaz
- varchar naproti tomu se ukládá přesně tak, jak je, tzn. že případné mezery JSOU signifikantní
- char neumožňuje uložit např. byte s hodnotou 0

Datové typy PostgreSQL

- binární
 - `bytea` – řetězec bytů proměnné délky
 - analogické datovému typu BLOB z SQL standardu
 - některé znaky se musí zadávat pomocí tzv. escape sekvencí ve tvaru

`E'\\000'`

Datové typy PostgreSQL

- datum a čas
 - `timestamp` – časové razítko
 - `time`
 - `interval`
- umožňují též uchovávat informaci o časovém pásmu
- různé formáty data a času
 - 1999-01-08, January 8, 1999, 1/8/1999, 990108,...
- speciální hodnoty
 - `now`
 - `today`, `tomorrow`, `yesterday`
 - `infinity`, `-infinity`
 - ...

Datové typy PostgreSQL

- boolean

- boolean

- kromě TRUE/FALSE lze použít rovněž

- t/f

- y/n

- yes/no

- on/off

- 1/0

Datové typy PostgreSQL

- výčtové typy

- enum

- nutno je vytvořit, poté se použijí dle jména (mood)
`CREATE TYPE mood AS ENUM ('sad', 'ok', 'happy');`
 - je definováno uspořádání, a to podle pořadí, v jakém byly prvky definovány
 - i když více výčtových typů obsahuje tentýž identifikátor, nelze porovnávat relačními operátory

Datové typy PostgreSQL

- geometrické typy
 - point
 - line
 - box
 - circle
- 2D objekty
- bitové řetězce
 - posloupnosti 0 a 1
 - bit(n)
 - bit varying(n)

```
INSERT INTO test VALUES (B'101', B'00');
```

Datové typy PostgreSQL

- pole

- []

- lze použít skoro na jakýkoli datový typ: `text[]`
 - velmi mnoho možností použití

```
CREATE TABLE sal_emp (  
    name text,      pay_by_quarter integer[],    schedule text[]  
);
```

```
INSERT INTO sal_emp  
VALUES ('Bill',  
    ARRAY[10000, 10000, 10000, 10000],  
    '{"meeting", "lunch"}, {"training", "presentation"}');
```

```
SELECT pay_by_quarter[2:3] FROM sal_emp;
```

Datové typy PostgreSQL

- kompozitní typy
 - obdoba tabulky
 - velmi mnoho možností použití
 - přístup k prvkům tečkovou notací
 - jeden „kompozitní“ řádek lze *ad hoc* zkonstruovat příkazem ROW

```
ROW(1, 2.5, 'this is a test');
```

```
CREATE TYPE complex AS (  
    r double precision,  
    i double precision  
);
```

Přetypování

- datové typy lze do jisté míry převádět jeden na druhý
 - implicitní konverze
 - explicitní konverze
 - CAST (expression AS type)
 - expression::type

```
CAST ( 12.0 AS text );  
12.0::text;
```

Dostupné funkce a operátory

- existují myriády vestavěných funkcí a operátorů pro všemožné operace nad datovými typy
 - logické (AND, OR, NOT)
 - relační (<, >, =, >=, <=, <> nebo !=)
 - x [NOT] BETWEEN a AND b (vč. okrajů intervalu)
 - x IS [NOT] NULL
 - x IS [NOT] { TRUE | FALSE | UNKNOWN }
 - matematické (+, -, *, /, % modulo, ^ mocnina, ! fact, ...)
 - bitové (&, |, # xor, ~ not, <<, >> posun)
 - matematické funkce
 - abs, random, exp, div, log, pi, sqrt, sign, sin, cos,
 - řetězcové funkce a operátory
 - s1 || s2 zřetězení (může být jeden neřetězec)
 - length, substring, overlay, chr, replace, strpos,

Dostupné funkce a operátory

- operátory a funkce nad binárními řetězci
- operátory nad bitovými řetězci (&, |, #, ~, <<, >>)
- podobnost
 - str [NOT] LIKE maska (zástupné znaky % a _)
 - str [NOT] SIMILAR TO regexp (| alt, * rep 0-∞, + rep 1-∞)
- mnoho možností pro vyhodnocení regulárních výrazů
- funkce pro formátování data a času
- funkce pro XML
- funkce pro síťové adresy
- funkce a operátory pro datum a čas
 - sčítání, odečítání, násobení, dělení
 - now, current_date, current_time, timeofday, ...
 - extract:

```
SELECT EXTRACT(YEAR FROM TIMESTAMP '2001-02-16 20:38:40');
```

Dostupné funkce a operátory

- funkce pro operace nad ENUM
- funkce a operátory pro geometrické typy
 - translace, rotace, změna měřítka, vzdálenost, překryv, vzájemná poloha, rovnoběžnost, ...
 - plocha, výška, šířka, typ cesty, ...
- funkce a operátory nad poli
 - porovnávání (lexikograficky), test podmnožiny, test průniku, zřetězení (`||`), `array_fill`, `array_length`, `unnest`, `generate_subscripts`, ...
- systémové funkce
 - `current_database`, `user`, `version`, ...
 - zjišťování práv a spousta dalších věcí

Agregační funkce

- na seskupenou tabulku (GROUP BY) lze aplikovat agregační funkce

```
SELECT MAX(délka) AS 'Nejdelší' FROM Silnice  
SELECT COUNT(DISTINCT délka) FROM Silnice
```

- avg, max, min, count, sum, bool_and, bool_or
- další statistické funkce (stddev, variance, corr,...)
- pozor na hodnoty NULL a nulový počet řádků
 - když je jako vstup agregační funkce použit nulový počet řádků, count(*) vrátí 0, ale sum(x) vrátí NULL!

Podmíněné výrazy

- neprocedurální vyhodnocení podmínek, např. pro zajištění IO
- CASE – běžné podmíněné větvení

```
SELECT a,  
       CASE WHEN a=1 THEN 'one'  
            WHEN a=2 THEN 'two'  
            ELSE 'other'  
       END
```

```
FROM test;
```

----- nebo

```
SELECT a,  
       CASE a WHEN 1 THEN 'one'  
            WHEN 2 THEN 'two'  
            ELSE 'other'  
       END
```

```
FROM test;
```

Podmíněné výrazy

■ COALESCE

- vybere ze seznamu první hodnotu, která není NULL
- řešení případů, kdy očekáváme NULL
- pozor na nutnost přetypování
 - všechny možné hodnoty v COALESCE musí mít stejný datový typ

COALESCE(Vypujcky.Cena::text, "Cena není určena");

klauzule LIMIT a OFFSET

- PostgreSQL má možnost, jak vrátit jen podmnožinu řádků dotazu

```
SELECT select_list  
      FROM table_expression  
      [ ORDER BY ... ]  
      [ LIMIT nlim ] [ OFFSET noffs ]
```

- vrací nejvýše *nlim* řádků dotazu počínaje *noffs*+1 řádkem
- pro *noffs*=0 je jako kdyby nebylo OFFSET zadáno
- je důležité použít řazení, aby měl příkaz smysl, protože obecně je pořadí řádků nepředvídatelné

Pohledy

- pohledy jsou virtuální (dynamické) tabulky
 - umožňují pohled na databázi ušít na míru uživateli
 - odstínění nežádoucích dat, odstínění změn DB
 - předvypočtení souhrnů
 - především pro dotazy, někdy je lze i aktualizovat
 - lze používat pro konstrukci dalších pohledů

```
CREATE VIEW Prazaci AS
```

```
    SELECT * FROM Zakaznici
```

```
    WHERE adresa LIKE '%Praha%';
```

```
CREATE OR REPLACE VIEW PoctyVypujcek AS
```

```
    SELECT rc, COUNT(c_kopie) FROM Vypujcky
```

```
    GROUP BY rc;
```

Dědičnost

- tabulka může být vytvořena jako odvozenina (potomek) jiné tabulky (rodičovské, mateřské)
 - potomek dědí
 - všechny sloupce rodiče
 - některá integritní omezení: check, not null
 - potomek nedědí
 - zbývající IO: unique, primary key, foreign key
 - přístupová práva

Dědičnost

- tabulka může být vytvořena jako odvozenina (potomek) jiné tabulky (rodičovské, mateřské)
 - záludnosti
 - dědičnost nezajistí žádnou pevnou vazbu mezi tabulkami, jako např. unikátní primární klíč přes sjednocení záznamů (tj. nelze tím automaticky zajistit realizaci entitního podtypu)
 - vkládání dat (INSERT) se děje jen na konkrétní tabulku, čili vkládání do rodiče nezajistí automaticky vložení do potomka

Dědičnost

-- databáze měst, z nichž chceme ještě vydělit hlavní města

```
CREATE TABLE Cities ( name text,  
                        population float,  
                        altitude int );
```

```
CREATE TABLE Capitals ( state text ) INHERITS (cities);
```

-- vybere záznamy z tabulky cities i capitals (tj. vč. potomků)

```
SELECT name, altitude FROM cities WHERE altitude > 500;
```

-- vybere záznamy jen z tabulky cities

```
SELECT name, altitude FROM ONLY cities WHERE ...;
```


Schemata

- databáze je členěna do hierarchické struktury
 - nejvyšší je cluster
 - cluster obsahuje pojmenované databáze
 - databáze obsahuje pojmenovaná schemata
 - schema obsahuje tabulky a další objekty
 - dvě schemata mohou obsahovat tabulku stejného jména
 - schemata nejsou tak striktně oddělena jako databáze, uživatel (připojí-li se k DB) může používat všechna, má-li na to oprávnění
 - schemata není možné vnořovat

Schemata

- schemata
 - pomáhají oddělovat různé skupiny uživatelů
 - člení databázi na logické celky
 - umožňují uchovávat objekty třetích stran (aplikace apod.), aby nekolidovaly identifikátory
 - vytváření a rušení:

```
CREATE SCHEMA myschema;
```

```
CREATE SCHEMA myschema AUTHORIZATION username;  
--majitelem nového schematu bude username
```

```
DROP SCHEMA myschema [CASCADE];
```

```
--CASCADE nařídí odstranění všech vnořených objektů
```

Schemata

- schemata
 - implicitně jsou všechny objekty umístěny ve schematu *public*
 - přístup k vnořeným objektům přes tečkovou notaci

```
CREATE TABLE myschema.mytable (...);
```

- tečková notace je otravná, proto se často nepíše - bez jejího použití prohledá stroj podle nějakého algoritmu seznam schemat (search path) a první vyhovující tabulka se vezme

```
SHOW search_path;
```

```
SET search_path TO myschema, public;
```

Schemata

- schemata
 - uživatel může pracovat jen se schematem, které vlastní, nebo je nutné oprávnění **USAGE** (viz přednáška o oprávněních)
 - search path implicitně začíná uživatelským schematem
 - standardně, všichni mají oprávnění **CREATE** a **USAGE** na schema PUBLIC, to lze změnit

REVOKE CREATE ON SCHEMA public FROM PUBLIC;
 - systémové schema **pg_catalog** – obsahuje systémové objekty

Schemata

- způsoby použití
 - ponechat pouze schema public
 - emuluje situaci, kdy schemata vůbec nejsou
 - pro každého uživatele vytvořit jeho schema
 - jelikož se search path prohledává od začátku, uživatel bude pracovat (bez tečkové notace) s vlastním sch.
 - pro instalované aplikace vytvořit vlastní schema
 - nutno zajistit patřičná oprávnění
 - uživatelé buď mohou používat kvalifikovaná jména (tečkovou notaci) nebo si zařadit schema do své search path